

DeepCube CFOP Solver - Project Phases

Overview

Train 4 separate neural heuristics (Cross, F2L, OLL, PLL) and chain them with A* search to solve Rubik's cubes.

Phase 1: State Representation (1 hour)

Goal: Build utility functions to convert cube state to tensor format.

Work:

- Flatten 54 stickers (6 faces $\times$ 3 $\times$ 3) to 54-dim vector
- Write helper functions for state encoding/decoding
- Test on known cube states

Output: `cube_utils.py`

Assign: Person A

Phase 2: Data Generation (4 hours wall time)

Generate training datasets for each CFOP phase. **Run 2a+2b in parallel, then 2c+2d in parallel.**

2a: Cross Data (2 hours) — Person A

- Scramble cubes randomly (depths 1-15 moves)
- Use BFS from scrambled state $\rightarrow$ solved cube
- Extract states where only cross layer is solved
- Compute distance to "cross solved"
- **Output:** `cross_data.h5` (50k+ examples)

2b: F2L Data (2 hours) — Person B (in parallel with 2a)

- Scramble cubes with cross already solved
- BFS from scrambled F2L state $\rightarrow$ solved
- Extract F2L-specific states
- Compute distance to "F2L solved"
- **Output:** `f2l_data.h5` (50k+ examples)

2c: OLL Data (2 hours) — Person A (after 2a done)

- Scramble cubes with cross + F2L solved
- BFS to solved
- Extract OLL states, distances

- **Output:** `oll_data.h5` (30k+ examples)

2d: PLL Data (2 hours) — Person B (after 2b done)

- Scramble cubes with cross + F2L + OLL solved
 - BFS to solved
 - Extract PLL states, distances
 - **Output:** `p11_data.h5` (20k+ examples)
-

Phase 3: Train Heuristic Networks (4 hours wall time)

Train 4 separate neural networks to predict distance-to-solved for each phase. **Run networks in parallel.**

3a: Cross Network (1.5 hours) — Person A

- Input: cube state (54-dim)
- Output: predicted distance (1 number)
- Loss: MSE
- Data: `cross_data.h5`
- **Output:** `cross_net.pt`

3b: F2L Network (2 hours) — Person B (in parallel with 3a)

- Input: cube state
- Output: predicted distance
- Loss: MSE
- Data: `f2l_data.h5`
- **Output:** `f2l_net.pt`

3c: OLL Network (1.5 hours) — Person A (after 3a done)

- Input: cube state
- Output: predicted distance
- Loss: MSE
- Data: `oll_data.h5`
- **Output:** `oll_net.pt`

3d: PLL Network (1.5 hours) — Person B (after 3b done)

- Input: cube state
 - Output: predicted distance
 - Loss: MSE
 - Data: `p11_data.h5`
 - **Output:** `p11_net.pt`
-

Phase 4: A* Search Implementation (2.4 hours wall time)

Implement A* search using each trained heuristic. **Run in parallel.**

4a: Cross A* Solver (1 hour) — Person A

- Implement A* algorithm
- Use `cross_net.pt` as heuristic
- Input: scrambled cube
- Output: sequence of moves to solve cross
- **Output:** `cross_solver.py`

4b: F2L A* Solver (1.2 hours) — Person B (in parallel with 4a)

- Implement A* algorithm
- Use `f2l_net.pt` as heuristic
- Input: cube with solved cross
- Output: sequence of moves to solve F2L
- **Output:** `f2l_solver.py`

4c: OLL A* Solver (1 hour) — Person A (after 4a done)

- Implement A* algorithm
- Use `oll_net.pt` as heuristic
- Input: cube with solved cross + F2L
- Output: sequence of moves to solve OLL
- **Output:** `oll_solver.py`

4d: PLL A* Solver (0.8 hours) — Person B (after 4b done)

- Implement A* algorithm
 - Use `pll_net.pt` as heuristic
 - Input: cube with solved cross + F2L + OLL
 - Output: sequence of moves to solve PLL
 - **Output:** `pll_solver.py`
-

Phase 5: Chain CFOP Solver (1 hour)

Goal: Pipe outputs from Cross → F2L → OLL → PLL into a single solver.

Work:

- Load all 4 `.pt` files
- Create pipeline: `solve_cross()` → `solve_f2l()` → `solve_oll()` → `solve_pll()`
- Handle state transitions between phases
- Return full move sequence

Output: `cfop_solver.py`

Assign: Person A

Phase 6: Testing & Benchmarking (1.5 hours)

Goal: Validate the solver and collect metrics.

Work:

- Generate 100 random scrambles (depths 5–30 moves)
- Run full CFOP solver on each
- Measure:
 - Success rate (% solved)
 - Average moves per phase
 - Total moves to solve
 - Solve time per scramble
- Compare to baselines (if applicable)
- Document results

Output:

- `test_results.txt` (metrics)
- `RESULTS.md` (write-up for resume)

Assign: Both (pair programming)

Summary Timeline

Phase	Duration	Parallelization
Phase 1	1h	—
Phase 2	4h	2a+2b parallel, 2c+2d parallel
Phase 3	4h	All 4 networks parallel
Phase 4	2.4h	All 4 A* solvers parallel
Phase 5	1h	—
Phase 6	1.5h	Pair
Total	~15h wall time	High parallelization

File Structure

```
rubiks-cfop/  
├─ cube.py                (your Cube class from notations.py)  
├─ cube_utils.py          (Phase 1: flatten, encode/decode)  
├─ data_gen.py            (Phase 2: generate datasets)  
├─ network.py            (Phase 3: network architecture)  
├─ train.py              (Phase 3: training loop)  
├─ cross_solver.py        (Phase 4a: Cross A*)  
├─ f2l_solver.py          (Phase 4b: F2L A*)  
├─ oll_solver.py          (Phase 4c: OLL A*)  
├─ pll_solver.py          (Phase 4d: PLL A*)  
├─ cfop_solver.py         (Phase 5: chain all phases)  
├─ test.py               (Phase 6: validation)  
├─ models/  
│   └─ cross_net.pt  
│   └─ f2l_net.pt  
│   └─ oll_net.pt  
│   └─ pll_net.pt  
├─ data/  
│   └─ cross_data.h5  
│   └─ f2l_data.h5  
│   └─ oll_data.h5  
│   └─ pll_data.h5  
└─ RESULTS.md            (Phase 6: final report)
```

Dependencies

- Python 3.8+
- PyTorch
- NumPy
- h5py
- tqdm

Key Decisions

- **CFOP approach:** Mimics human solving, smaller sub-problems
- **Separate networks:** One heuristic per phase for modularity
- *A search**: Optimal moves given good heuristic
- **h5py datasets:** Efficient storage for 100k+ examples
- **Parallel execution:** Both team members work simultaneously when possible